Práctica 3: Divide y Vencerás - Parte 2

Por
Javier Acosta Portocarrero - alu0101660769
Gabriel Gallardo Noda - alu0101633961

Diseño y Análisis de Algoritmos
Escuela Superior de Ingeniería y Tecnología

Índice

Índice **2**
Estructura **3**
Contenido de la clase AlgoritmoAproximadoPlanificación **6**
AlgoritmoAproximadoPlanificacion::Divide 6
AlgoritmoAproximadoPlanificacion::Combine 6
AlgoritmoAproximadoPlanificacion::EncontrarPeorDiaTrabajdoNoMinimo 7
AlgoritmoAproximadoPlanificacion::EncontrarPeorDiaTrabajado 7
AlgoritmoAproximadoPlanificacion::SolveSmall 7
AlgoritmoAproximadoPlanificacion::IsSmall 8
Contenido de la clase AlgoritmoVorazPlanificacionDia **8**
AlgoritmoVorazPlanificacionDia::Solve 8
AlgoritmoVorazPlanificacionDia::EncontrarMejorEmpleadoDisponible 8
Experimentación **9**
Experimentación 1 Días 7 Empleados 10 Turnos 6 9
Experimentación2_dias14_empleados10_turnos6 9
Experimentación3_dias30_empleados20_turnos10 10
Conclusión **11**

Estructura

En primer lugar, comentaremos cómo hemos estructurado esta parte de la práctica, la cual se organiza en dos carpetas separadas por su lógica y contenido:

La carpeta **/lib** contiene todo lo relacionado al algoritmo de divide y vencerás especificado en la primera parte de este proyecto y con la las clases usadas en la primera práctica de la asignatura, además de estar formateado y planteado siguiendo los principios SOLID y el patrón estrategia, de la siguiente manera:

- Primero se han creado las respectivas clases que permiten representar los componentes (entrada/salida) de nuestro algoritmo, es decir, la instancia y la solución, además de la clase abstracta algoritmo, que a partir de una instancia genera una solución (a partir del método virtual puro Solve).

- algoritmo_divide_y_venceras.h_: Se establece una clase con las propiedades (métodos) típicas de un algoritmo divide y vencerás que hereda de Algoritmo, teniendo un **método plantilla** llamado Solve (implementado en Algoritmo_divide_y_venceras.cc), que a partir de una instancia, realizando el típico procedimiento de dividir el problema hasta que sea lo suficientemente manejable para luego combinar todas las soluciones, da lugar a la solución de la instancia original. Todo algoritmo divide y vencerás deberá implementar los métodos específicos: Divide, Combine, IsSmall, SolveSmall. Estos métodos son virtuales puros.

Por otro lado la carpeta **/planificación_empleados** contiene todo lo relacionado con el problema de la planificación de una plantilla de empleados para realizar un determinado trabajo, siguiendo los principios SOLID, el patrón estrategia y el patrón plantilla, las clases implementadas son las siguientes:

- InstanciaPlanificaciónEmpleados: En esta clase que hereda de la clase Instancia de Algoritmo, nos encargamos de establecer las características típicas de dicha planificación (abordadas en el guión del proyecto, es decir, empleados, turnos, días a planificar, satisfacción, entre otros…) y su método de acceso (Getters), por otro lado en su implementación (Instancia_planificación_empleados.cc) se encuentra el constructor de dicho objeto de planificación y Getters a otros atributos.

- CargaInstanciaPlanificaciónjson: Esta clase tiene como interfaz (clase abstracta) a la clase CargaInstanciaPlanificación para seguir los principios SOLID, además recibe la ruta a un fichero en el constructor, puesto que en el procedimiento CargaInstanciaPlanificacionJson::Load de la implementación (Carga_instancia_planificación_json.cc) se procesará cada característica de planificación del fichero de entrada para finalmente generar una objeto de la clase InstanciaPlanificaciónEmpleados que reunirá las características finales, el cual devolverá.

- AlgoritmoAproximadoPlanificación: Esta clase heredera de Algoritmo_divide_y_venceras_, reescribirá todos sus métodos menos Solve ya que lo mantendrá para usarlo como se mencionó anteriormente, además que también permitirá la búsqueda del peor día de trabajo en donde se sobrepase el mínimo y cuando no se sobrepasa. Más adelante se tratará en profundidad todo el contenido de esta clase.

- AlgoritmoVorazPlanificacionDia: Esta clase heredera de la clase Algoritmo (Algoritmo), reescribe su método Solve para resolver la instancia cuando es de tamaño 1 y permite la búsqueda del mejor empleado disponible para un turno y día específicos, a través del método EncontrarMejorEmpleadoDisponible. Más adelante se tratará en más profundidad el contenido de esta clase.

- SolucionPlanificacionEmpleados: Esta clase construye y moldea la solución adquirida de nuestro algoritmo para la planificación de empleados, presentando no solo sus respectivos campos (empleados, turnos, días a planificar, entre otros …) sino también sus métodos de acceso, al igual que otros métodos que aportan otros datos relacionados con dichos campos y finalmente el método que calcula el valor objetivo de esta solución. (implementación en Solución_planificación_empleados.cc). Además de tener los atributos de la clase de instancia, también tiene atributos para la cantidad de días trabajados de cada empleado, además de una estructura de datos de booleanos que indica que turnos de qué días ha trabajado cada empleado.

- PlanificadorEmpleados: Esta clase se encarga de resolver la instancia del problema llamando al Solve de la estrategia (clase Algoritmo) adecuada en el método Planificar, además se permite cambiar de estrategia con el método SetEstrategia (implementación en Planificador_empleados.cc). Siguiendo el patrón estrategia, esta clase sería el contexto.

- AlmacenarSolucionPlanificacionFichero: Esta clase, que recibe en su constructor la ruta a un fichero, tiene como interfaz (clase abstracta) a AlmacenarSolucionPlanificacion siguiendo los principios SOLID y el patrón de estrategia, reescribiendo el método Almacenar para guardar y representar la información de la solución adjunta, en un formato sencillo y legible (implementación en Almacenar_solucion_planificacion_fichero.cc).

Contenido de la clase AlgoritmoAproximadoPlanificación

A continuación se tratarán los distintos métodos de la clase, indicando el razonamiento seguido, pero si se quiere profundizar más, en el código al que llevan los enlaces hay comentarios más exhaustivos de todos los procedimientos llevados a cabo, sus motivos y el por qué del orden en el que se realizan. Lo mismo se realizará para la clase del algoritmo voraz para instancias de 1 día.

AlgoritmoAproximadoPlanificacion::Divide

En Algoritmo_aproximado_planificación.cc se implementa el método Divide, implementación enfocada en dividir en dos instancias los datos de la instancia original por el número de días a planificar por la izquierda (desde el menor día hasta la mitad) y por la derecha (el resto de días), y rellenar sus respectivos datos, finalmente ambas instancias de planificación de empleados son guardadas y devueltas en un vector. Recalcar que hay datos comunes en ambas instancias, como la cantidad de empleados o de turnos y la cantidad de días original.

AlgoritmoAproximadoPlanificacion::Combine

En Algoritmo_aproximado_planificación.cc se implementa el método Combine, implementación enfocada en que, a partir de la división anterior en base a los días de la izquierda y derecha, se crea una solución con la fusión de los campos locales (satisfacción, mínimo de empleados, cantidad de días) recorriendo ambas colecciones de datos y sus respectivos aspectos, y tras esto se comprueba si la cantidad de días de la fusión es igual a la cantidad de días original del problema, y, si es así, se restauran las restricciones del problema, es decir, para cada empleado se van eliminando turnos de trabajo hasta que se cumplan los días mínimos de descanso.

Importante recalcar, que para conseguir una mejor solución, primero se eliminan los turnos trabajados en los que hay empleados “sobrantes”, y cuando ya no quedan turnos de este tipo, se eliminan turnos de trabajo del empleado donde se rompe la condición de empleados mínimos.

AlgoritmoAproximadoPlanificacion::EncontrarPeorDiaTrabajdoNoMinimo

En Algoritmo_aproximado_planificación.cc se implementa el método EncontrarPeorDiaTrabajdoNoMinimo, el cual recibe un empleado y una solución, y busca el día en el que este empleado trabaja (TrabajaEmpleadoDiaTurno) y se sobrepasa el mínimo (GetMinimoEmpleados, EmpleadosTurno), con el nivel de satisfacción más reducido (GetSatisfaccion), devolviendo un std::pair<int, int> que incluye el número del día y turno encontrado. Es decir. básicamente encuentra el peor turno posible en el que trabajo un empleado especificado y en el que hay empleados “sobrantes”

AlgoritmoAproximadoPlanificacion::EncontrarPeorDiaTrabajado

En Algoritmo_aproximado_planificación.cc se implementa EncontrarPeorDiaTrabajado, método muy parecido al exterior, que recibe los mismos parámetros, y que busca el día en el que un empleado trabaja (TrabajaEmpleadoDiaTurno), con el nivel de satisfacción más reducido (GetSatisfaccion), devolviendo un std::pair<unsigned, unsigned> que incluye el número del día y turno encontrado. Es decir, busca el peor turno en el que trabaja un empleado, pero sin tener en cuenta que ese día tenga empleados “sobrantes”.

AlgoritmoAproximadoPlanificacion::SolveSmall

En Algoritmo_aproximado_planificación.cc se implementa SolveSmall, método que recibe una instancia y que se encarga de llamar al algoritmo voraz (algoritmo_small_ de clase Algoritmo) para resolver la instancia cuando es de tamaño 1, es decir, tiene solo un día o menos. Finalmente devuelve un puntero de la clase Solución.

```C/C++
Solucion* AlgoritmoAproximadoPlanificacion::SolveSmall(Instancia*
entrada) {
  Solucion* solucion = algoritmo_small_->Solve(entrada);
  return solucion;
}
```

AlgoritmoAproximadoPlanificacion::IsSmall

En Algoritmo_aproximado_planificación.cc se implementa IsSmall, método que recibe una instancia y que se encarga de comprobar y retornar que el tamaño de está sea 1, es decir, que su cantidad de días sea menor o igual 1.

```
C/C++
bool AlgoritmoAproximadoPlanificacion::IsSmall(Instancia* entrada) {
  InstanciaPlanificacionEmpleados* entrada_procesada =
dynamic_cast<InstanciaPlanificacionEmpleados*>(entrada);
  // Lo consideramos pequeño cuando la cantidad de días es 1 o menos,
  return (entrada_procesada->GetCantidadDias() <= 1);
}
```

Contenido de la clase AlgoritmoVorazPlanificacionDia

AlgoritmoVorazPlanificacionDia::Solve

En Algoritmo_voraz_planificacion_dia.cc se implementa el método Solve, el se ejecuta cuando la instancia es de tamaño 1, este recibe como parámetro una instancia que debe ser de tamaño 1,crea una solución con los datos pertinentes, y hace dos pasadas, una para asignar a cada turno la cantidad mínima de empleados, siempre asignado el empleado con mejor satisfacción posible, y otra pasada asignando a cada turno el mejor empleado posible hasta que no quede ningún empleado disponible.

AlgoritmoVorazPlanificacionDia::EncontrarMejorEmpleadoDisponible

En Algoritmo_voraz_planificacion_dia.cc se implementa el método EncontrarMejorEmpleadoDisponible, el cual se recibe por parámetro una solución, un día y un turno, y busca el mejor empleado posible para ese turno que no haya trabajado ese día, y, si no existe ningún empleado disponible, devuelve -1 para indicarlo, en caso contrario devuelve el identificador del empleado.

Experimentación

A continuación se tratarán los resultados obtenidos con 3 de las instancias propuestas en el aula virtual, teniendo cada una una cantidad de días, empleados y turnos distintos. Más adelante se realizará una experimentación más profunda.

Experimentación 1 Días 7 Empleados 10 Turnos 6

En esta primera instancia se planifican 7 días, con 10 empleados disponibles y 6 turnos por día. El algoritmo genera una planificación válida respetando en todo momento la restricción obligatoria de días mínimos de descanso para cada empleado.

En este caso, el número de empleados es relativamente reducido si se tiene en cuenta que algunos de ellos deben cumplir días obligatorios de descanso, y también que cada empleado solo puede trabajar en un turno por día. Como consecuencia, la planificación resulta bastante ajustada y en ciertos turnos el número de empleados asignados coincide exactamente con el mínimo requerido.

A pesar de esta limitación, el algoritmo consigue generar una planificación equilibrada y mantener un buen nivel de satisfacción general. El valor de la función objetivo obtenido es 4684, lo que indica que las asignaciones realizadas presentan niveles de satisfacción razonablemente altos.

Experimentación2_dias14_empleados10_turnos6

En esta instancia el algoritmo mantiene el cumplimiento de los días mínimos de descanso de los empleados tras el proceso de combinación de subsoluciones. Gracias al mayor número de días, el algoritmo puede aprovechar más a los empleados, asignando mejores satisfacciones en los trabajos. Se siguen respetando los empleados mínimos de los turnos, pero esto se debe solo a que la cantidad de empleados es suficiente para hacerlo.

Como consecuencia, el valor de la función objetivo aumenta con respecto a la instancia anterior, consiguiendo un valor de 9385, gracias a que al haber más turnos cubiertos (al haber más en total), y no aumentar demasiado los días de descanso en comparación, se aumenta este significativamente.

Experimentación3_dias30_empleados20_turnos10

En la tercera instancia el tamaño del problema aumenta significativamente. En este caso se planifican 30 días, con 20 empleados disponibles y 10 turnos por día.

Este incremento en el tamaño del problema supone un mayor número de combinaciones posibles y un mayor número de asignaciones que deben ser gestionadas por el algoritmo. Sin embargo, gracias al enfoque de Divide y Vencerás, el problema se divide recursivamente en subproblemas más pequeños que pueden resolverse de forma eficiente mediante el algoritmo voraz.

A pesar del aumento considerable en el tamaño del problema, el algoritmo sigue siendo capaz de generar soluciones válidas en tiempos razonables (prácticamente instantáneo en el pórtatil en el que lo hemos ejecutado), lo que demuestra la escalabilidad del enfoque utilizado. El valor objetivo aumenta considerablemente debido a que hay más turnos que pueden ser cubiertos, lo cual influye muy positivamente en este.

Experimentación límite

A parte de probar nuestro algoritmo con las instancias del campus, para las cuales, gracias a las varias pasadas que hacemos tanto en el algoritmo voraz como a las que se hacen en el combine del algoritmo DYV, nuestro programa proporciona soluciones de muy buena calidad, hemos querido probar también con instancia más extremas, para ver cómo se comporta el algoritmo en estas situaciones.

Para esto, hemos partido de la primera instancia del campus, y hemos probado a aumentar la cantidad de días de descanso de los empleados. Los resultados originales para los días 3, 4 y 5 eran los siguientes:

Dia 3

Empleado	T0	T1	T2
Empleado 0	0	X	0
Empleado 1	0	0	X
Empleado 2	0	0	0
Empleado 3	X	0	0
Empleado 4	X	0	0

Dia 4

Empleado	T0	T1	T2
Empleado 0	0	0	X
Empleado 1	0	0	0
Empleado 2	0	X	0
Empleado 3	X	0	0
Empleado 4	0	0	0

Dia 5

Empleado	T0	T1	T2
Empleado 0	0	0	X
Empleado 1	X	0	0
Empleado 2	0	X	0
Empleado 3	0	0	0
Empleado 4	X	0	0

Como se puede ver, se cubren siempre los empleados mínimos para cada turno, que en este caso es siempre 1, y, además, como sobran turnos de trabajo por los empleados, se asignan empleados adicionales a los turnos para maximizar el valor objetivo. El valor objetivo conseguido es de 2308.

Si aumentamos los días de descanso para todos los empleados de forma moderada, vemos que la solución pasa a ser la siguiente:

Dia 3

Empleado	T0	T1	T2
Empleado 0	0	X	0
Empleado 1	0	0	X
Empleado 2	0	0	0
Empleado 3	X	0	0
Empleado 4	0	0	0

Dia 4

Empleado	T0	T1	T2
Empleado 0	0	0	X
Empleado 1	0	0	0
Empleado 2	0	0	0
Empleado 3	X	0	0
Empleado 4	0	X	0

Dia 5

Empleado	T0	T1	T2
Empleado 0	0	0	X
Empleado 1	X	0	0
Empleado 2	0	X	0
Empleado 3	0	0	0
Empleado 4	0	0	0

Como se puede apreciar, el algoritmo se adapta a las nuevas condiciones, y elimina los empleados adicionales de los turnos, de forma que aún sea posible mantener la cantidad mínima de empleados para cada turno. Es decir, gracias a cómo hemos estructurado el algoritmo, este es capaz de adaptarse a condiciones más extremas sin que haya un

gran impacto al valor objetivo, pues a pesar de aumentar bastante los días de descanso, el valor solo desciende a 2250.

Ahora veremos qué pasa si seguimos aumentando los días de descanso:

Dia 3

Empleado	T0	T1	T2
Empleado 0	0	X	0
Empleado 1	0	0	X
Empleado 2	0	0	0
Empleado 3	X	0	0
Empleado 4	0	0	0

Dia 4

Empleado	T0	T1	T2
Empleado 0	0	0	X
Empleado 1	0	0	0
Empleado 2	0	0	0
Empleado 3	X	0	0
Empleado 4	0	0	0

Dia 5

Empleado	T0	T1	T2
Empleado 0	0	0	X
Empleado 1	X	0	0
Empleado 2	0	X	0
Empleado 3	0	0	0
Empleado 4	0	0	0

Como vemos, la cantidad de empleados y días de descanso ya no es “compatible”, y el algoritmo, por mucho que intenta distribuir los empleados por turno, ya no es capaz de cumplir, en este caso, los empleados mínimos para el turno 1 del día 4. Con esto se refleja que el algoritmo depende mucho de la instancia dada, pues a veces, la diferencia de un único día más de descanso, puede hacer que en un turno no haya ningún empleado.

Esto se refleja en el valor objetivo, que desciende bastante más que antes, a un valor de 2145, aunque la cantidad de descansos no es mucho mayor que en el caso anterior.

En estos enlaces se puede consultar la solución del primer caso y segundo caso en los que se han cambiado las condiciones de la instancia, en las que se puede comprobar el comportamiento descrito.

Conclusión

En general, el algoritmo desarrollado funciona bastante bien para las distintas instancias probadas. La combinación del enfoque de Divide y Vencerás con el algoritmo voraz, con dos pasadas para conseguir mejores soluciones parciales, para resolver los casos pequeños permite construir soluciones razonables de forma eficiente. Además, durante la fase de combinación se tienen en cuenta aspectos globales como los días mínimos de descanso de los empleados, solo cuando la cantidad de días es igual a la original del problema (si no se eliminarían demasiados turnos trabajados innecesariamente), garantizando que esta restricción se cumpla siempre, además, el realizar dos pasadas para reducir lo máximo posible los turnos con cantidad de empleados insuficiente también favorece la generación de mejores soluciones.

Destacar que tener en cuenta la función objetivo indicada ayuda mucho a que las soluciones obtenidas sean de buena calidad, ya que damos mucha importancia a cubrir el número mínimo de empleados (lo cual tiene un peso mucho mayor) en los turnos antes de maximizar la satisfacción de los trabajadores.

Sin embargo, al tratarse de un algoritmo aproximado basado en decisiones locales, en algunos casos pueden quedar turnos sin cubrir mientras que en otros hay más empleados de los necesarios, en los ejemplos vistos esto no es lo común, pero dependiendo del escenario puede pasar. Esto ocurre porque el algoritmo no realiza cambios o intercambios de turnos entre empleados una vez hecha la planificación inicial.

También se pueden generar soluciones no tan buenas porque, para algunas instancias, la cantidad de empleados es simplemente insuficiente, o la cantidad de descansos es excesiva. Aun así, los resultados obtenidos en las distintas instancias probadas muestran que el algoritmo se comporta de forma adecuada y consigue soluciones bastante buenas para el problema planteado.

Por último, mencionar que se ha comparado con varios compañeros los valores objetivos obtenidos, y nuestro programa siempre consigue resultados mejor, aunque la mejoría normalmente no es excesiva,